

赛区评阅编号（由赛区组委会填写）：

2026 高教社杯全国大学生数学建模竞赛

承 诺 书

我们仔细阅读了《全国大学生数学建模竞赛章程》和《全国大学生数学建模竞赛参赛规则》（以下简称“竞赛章程和参赛规则”，可从 <http://www.mcm.edu.cn> 下载）。

我们完全清楚，在竞赛开始后参赛队员不能以任何方式，包括电话、电子邮件、“贴吧”、QQ 群、微信群等，与队外的任何人（包括指导教师）交流、讨论与赛题有关的问题；无论主动参与讨论还是被动接收讨论信息都是严重违反竞赛纪律的行为。

我们完全清楚，在竞赛中必须合法合规地使用文献资料和软件工具，不能有任何侵犯知识产权的行为。否则我们将失去评奖资格，并可能受到严肃处理。

我们以中国大学生名誉和诚信郑重承诺，严格遵守竞赛章程和参赛规则，以保证竞赛的公正、公平性。如有违反竞赛章程和参赛规则的行为，我们将受到严肃处理。

我们授权全国大学生数学建模竞赛组委会，可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

我们参赛选择的题号（从 A/B/C/D/E 中选择一项填写）： C

我们的报名参赛队号（12 位数字全国统一编号）： 2026080901

参赛学校（完整的学校全称，不含院系名）： 哈尔滨工业大学（深圳）

参赛队员（打印并签名）：1. 成员 A

2. 成员 B

3. 成员 C

指导教师或指导教师组负责人（打印并签名）： 指导老师

（指导教师签名意味着对参赛队的行为和论文的真实性负责）

日期： 2026 年 08 月 09 日

（请勿改动此页内容和格式。此承诺书打印签名后作为纸质论文的封面，注意电子版论文中不得出现此页。以上内容请仔细核对，如填写错误，论文可能被取消评奖资格。）

赛区评阅编号：
(由赛区填写)

全国评阅编号：
(全国组委会填写)

2026 高教社杯全国大学生数学建模竞赛

编 号 专 用 页

赛区评阅记录（可供赛区评阅时使用）：

评阅人						
备注						

送全国评阅统一编号：

(赛区组委会填写)

(请勿改动此页内容和格式。此编号专用页仅供赛区和全国评阅使用，参赛队打印后装订到纸质论文的第二页上。注意电子版论文中不得出现此页。)

古代玻璃制品的成分分析与鉴别

摘要

本文针对古代玻璃制品的成分分析与鉴别问题，建立了系统的数学模型。首先，采用卡方检验分析表面风化与玻璃类型、纹饰和颜色的关系，并基于描述性统计揭示风化前后化学成分的变化规律；对于风化前的成分预测，建立了加权平均预测占比模型、多元线性回归模型和 K 近邻回归模型。其次，针对高钾玻璃和铅钡玻璃的分类规律，运用 K-means++ 聚类算法进行亚类划分，借助轮廓系数和肘部法则确定最优聚类数并验证合理性。再次，构建 K 近邻（KNN）分类模型对未知类别的玻璃文物进行类型鉴别，并通过交叉验证分析分类结果的敏感性。最后，采用相关性分析和对应分析方法挖掘不同类别玻璃化学成分之间的关联关系并比较其差异性。模型验证表明，所建方法能够有效处理成分数据，为考古研究提供定量分析工具。

关键字： 玻璃文物 风化预测 K-means++ 聚类 KNN 分类 对应分析

一、问题重述

丝绸之路是古代中西方文化交流的重要通道，玻璃作为早期贸易往来的宝贵物证，具有重要的考古学研究价值。古代玻璃的主要原料是石英砂，主要化学成分是二氧化硅(SiO_2)。由于纯石英砂熔点较高，炼制时需要添加助熔剂（如草木灰、天然泡碱、硝石、铅矿石等），并添加石灰石作为稳定剂。不同助熔剂导致玻璃的主要化学成分不同：铅钡玻璃中氧化铅(PbO)、氧化钡(BaO)含量较高，通常被认为是我国自己发明的玻璃品种；钾玻璃则以含钾量高的物质如草木灰作为助熔剂。

古代玻璃极易受埋藏环境影响而风化。在风化过程中，内部元素与环境元素进行大量交换，导致成分比例发生变化，从而影响对其类别的正确判断。现有一批我国古代玻璃制品的相关数据，已分为高钾玻璃和铅钡玻璃两种类型。附件表单 1 给出了文物的分类信息（纹饰、类型、颜色、表面风化），附件表单 2 给出了主要成分所占比例，附件表单 3 为未分类玻璃文物的化学成分数据。数据具有成分性特点，各成分比例累加和应为 100%，但因检测手段等原因可能导致累加和非 100%，本题将累加和介于 85%–105% 之间的数据视为有效数据。

需要解决以下四个问题：

1. 分析玻璃文物表面风化与玻璃类型、纹饰和颜色的关系；结合玻璃类型分析有无风化化学成分含量的统计规律；根据风化点检测数据预测风化前的化学成分含量。
2. 分析高钾玻璃、铅钡玻璃的分类规律；对每个类别选择合适的化学成分进行亚类划分，给出具体的划分方法及结果，并对分类结果的合理性和敏感性进行分析。
3. 对附件表单 3 中未知类别玻璃文物的化学成分进行分析，鉴别其所属类型，并对分类结果的敏感性进行分析。
4. 针对不同类别的玻璃文物样品，分析其化学成分之间的关联关系，并比较不同类别之间的化学成分关联关系的差异性。

二、模型的假设

1. 假设附件中所给数据均真实可靠，空白处表示未检测到该成分，可视为含量为零。
2. 假设成分比例累加和介于 85%–105% 之间的数据为有效数据，其余为无效数据。
3. 假设同一文物不同部位的采样点数据可独立用于分析。
4. 假设风化过程中化学成分的变化遵循一定的统计规律，可用于预测风化前的成分含量。
5. 假设未风化点的成分数据可代表文物风化前的原始成分。

三、符号说明

表 1 主要符号说明

符号	含义
x_{ij}	第 i 个样本第 j 种化学成分的含量
$\bar{x}_j$	第 j 种化学成分的均值
s_j	第 j 种化学成分的标准差
K	聚类数目
k	KNN 算法中的近邻个数
$d(\cdot, \cdot)$	欧氏距离
r_{pq}	第 p 种与第 q 种化学成分的相关系数
P	卡方检验的显著性水平
χ^2	卡方统计量
SSE	簇内误差平方和
$s(i)$	轮廓系数

四、数据预处理

4.1 数据清洗

根据题目要求，删除累加和不在 85%–105% 范围内的无效数据。经检验，编号为 15 和 17 的两条数据存在异常，予以剔除。将颜色为空值的部分根据风化程度规律进行处理，将其他空值填“0”处理。

4.2 数据变换

由于成分数据具有“定和约束”（各成分之和为 100%），直接使用原始数据进行统计分析可能产生伪相关等问题。为此，采用中心对数比变换（Center Log-Ratio Transfor-

mation, CLR) 将成分数据从单纯形空间映射到欧式空间。CLR 变换定义为:

$$y_{ij} = \ln \left(\frac{x_{ij}}{g(\mathbf{x}_i)} \right), \quad g(\mathbf{x}_i) = \left(\prod_{j=1}^p x_{ij} \right)^{1/p} \quad (1)$$

其中 $g(\mathbf{x}_i)$ 为第 i 个样本各成分的几何均值。变换后的数据在欧式空间中可进行常规的统计分析与建模。

五、问题一：表面风化分析与风化前成分预测

5.1 风化与类型、纹饰、颜色的关系分析

5.1.1 分析方法

针对玻璃类型、纹饰、颜色三个定类变量与表面风化（风化/无风化）之间的关系，采用卡方检验（Chi-Square Test）进行独立性分析。卡方检验的统计量为：

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \quad (2)$$

其中 O_{ij} 为观测频数， E_{ij} 为期望频数。根据显著性 P 值是否小于 0.05 判断是否存在显著性差异。

5.1.2 效应量化

在卡方检验的基础上，进行效应量化分析，包括 phi 系数、Cramér's V 系数、列联系数与 lambda 系数。

5.1.3 分析结果

（待完成：根据附件表单 1 数据进行卡方检验，计算各变量与风化的卡方统计量、 P 值及效应量，判断玻璃类型、纹饰、颜色与表面风化之间是否存在显著性差异。）

5.2 风化前后化学成分含量的统计规律

5.2.1 分析方法

分别针对高钾玻璃和铅钡玻璃两类，将样本按风化状态分为风化组与未风化组，对各主要化学成分进行描述性统计分析（均值、标准差、中位数等），并绘制频率分布直方图，观察数据的分布与波动情况。

5.2.2 统计规律

（待完成：计算高钾玻璃风化前后各化学成分的均值变化、标准差变化；计算铅钡玻璃风化前后各化学成分的均值变化、标准差变化。）

（待完成：绘制风化前后各主要化学成分的频率分布直方图，分析数据波动情况。）

5.3 风化前化学成分含量预测

5.3.1 加权平均预测占比模型

针对风化点检测数据的特殊性，建立加权平均预测占比模型。该模型基于同一类型玻璃中未风化点的成分数据，通过加权平均的方式预测风化点的风化前成分。

对于风化样本 i ，其第 j 种化学成分的风化前含量预测值为：

$$\hat{x}_{ij}^{\text{pre}} = \sum_{k \in \mathcal{U}_t} w_{ik} \cdot x_{kj}^{\text{unweathered}} \quad (3)$$

其中 $\mathcal{U}_t$ 为与样本 i 同类型的所有未风化样本集合， w_{ik} 为权重系数（可根据样本间相似度确定）， $x_{kj}^{\text{unweathered}}$ 为未风化样本 k 的第 j 种成分含量。

5.3.2 多元线性回归模型

采用多元线性回归模型，以风化后的成分含量为自变量，预测风化前的成分含量。以 SiO_2 为例，回归模型形式为：

$$\text{SiO}_2^{\text{pre}} = \beta_0 + \sum_j \beta_j \cdot x_j^{\text{weathered}} + \varepsilon \quad (4)$$

其中 $x_j^{\text{weathered}}$ 为风化后第 j 种化学成分的含量。

5.3.3 K 近邻回归模型

采用 K 近邻回归（K=5），以“风化后成分”为输入，“同类未风化成分均值”为参考，预测风化前的成分含量。对于待预测的风化样本，在同类未风化样本中寻找 K 个最近邻，取其成分含量的加权平均值作为预测值。

5.3.4 预测结果

（待完成：对表单 2 中风化点及严重风化点的各化学成分进行预测，给出风化前化学成分含量的具体数值。）

（待完成：比较加权平均模型、多元线性回归模型与 K 近邻回归模型的预测效果，选择最优模型。）

六、问题二：玻璃分类规律与亚类划分

6.1 分类规律分析

6.1.1 高钾玻璃与铅钡玻璃的区分特征

根据题目背景，高钾玻璃以 K_2O 含量高为特征（以草木灰为助熔剂），铅钡玻璃以 PbO 和 BaO 含量高为特征（以铅矿石为助熔剂）。通过对表单 2 数据的统计分析，识别两类玻璃在化学成分上的关键差异指标。

6.1.2 分析方法

计算两类玻璃各化学成分的均值与标准差，识别差异显著的成分指标。采用 t 检验或 Mann-Whitney U 检验验证差异的显著性。

（待完成：计算高钾玻璃与铅钡玻璃各化学成分的均值、标准差，识别关键区分指标。）

6.2 亚类划分方法

6.2.1 K-means++ 聚类算法

采用 K-means++ 聚类算法对每个类别进行亚类划分。K-means++ 通过优化初始聚类中心的选择，克服了传统 K-means 算法对初始值敏感的缺点。

算法步骤如下：

1. 从数据集中随机选择第一个聚类中心。
2. 对于每个样本，计算其与最近聚类中心的距离 $D(x)$ 。
3. 以正比于 $D(x)^2$ 的概率选择下一个聚类中心。
4. 重复步骤 2-3 直到选择出 K 个初始聚类中心。
5. 执行标准 K-means 算法迭代直至收敛。

6.2.2 最优聚类数确定

采用肘部法则（Elbow Method）和轮廓系数（Silhouette Coefficient）确定最优聚类数 K 。

肘部法则通过计算不同 K 值下的簇内误差平方和（SSE）：

$$SSE = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \quad (5)$$

选择 SSE 下降速度出现明显拐点处的 K 值。

轮廓系数的计算公式为：

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (6)$$

其中 $a(i)$ 为样本 i 到同簇其他样本的平均距离， $b(i)$ 为样本 i 到其他簇样本的最小平均距离。轮廓系数取值范围为 $[-1, 1]$ ，值越大表示聚类效果越好。

6.2.3 特征选择

针对高钾玻璃，选择 K_2O 、 SiO_2 、 Al_2O_3 、 CaO 等特征成分进行聚类。针对铅钡玻璃，选择 PbO 、 BaO 、 SiO_2 、 P_2O_5 等特征成分进行聚类。

6.3 亚类划分结果

（待完成：给出高钾玻璃的最优聚类数及亚类划分结果，列出每个亚类的样本编号及特征成分均值。）

（待完成：给出铅钡玻璃的最优聚类数及亚类划分结果，列出每个亚类的样本编号及特征成分均值。）

6.4 合理性与敏感性分析

6.4.1 合理性分析

通过以下方面验证亚类划分的合理性：

- 轮廓系数评估聚类紧密度与分离度。
- 各亚类间特征成分均值的差异显著性检验。
- 亚类结果与文物背景知识的一致性检验。

6.4.2 敏感性分析

通过以下方面分析分类结果的敏感性：

- 改变聚类数 K 观察分类结果的变化。
- 改变特征成分的组合观察分类结果的变化。
- 采用不同的聚类算法（如层次聚类）进行对比验证。

（待完成：给出敏感性分析的具体结果与结论。）

七、问题三：未知类别玻璃文物的类型鉴别

7.1 分类模型构建

7.1.1 K 近邻分类模型

采用 K 近邻 (KNN) 分类算法对表单 3 中未知类别的玻璃文物进行类型鉴别。KNN 算法的基本思想是：对于一个待分类样本，在训练集中找到与其最相似的 K 个样本（近邻），根据这 K 个近邻的类别标签通过投票法决定待分类样本的类别。

7.1.2 模型训练

将表单 2 中的有效数据作为训练集，分离特征列（化学成分）和标签列（类型）：

$$X = \text{化学成分数据}, \quad y = \text{类型标签（高钾/铅钡）} \quad (7)$$

采用 `train_test_split` 按 8:2 的比例划分训练集和测试集。

7.1.3 模型参数

采用 KNN 分类器，设置 $K = 1$ 。使用 `MultiOutputClassifier` 处理多输出问题。

7.2 分类结果

（待完成：对表单 3 中 A1–A8 共 8 个未知样本进行类型鉴别，给出每个样本的预测类型。）

7.3 敏感性分析

7.3.1 交叉验证

采用 K 折交叉验证 (K-Fold Cross Validation) 评估模型的稳定性。将训练数据分为 K 折，每次使用 $K - 1$ 折训练、1 折验证，重复 K 次，计算平均分类准确率。

7.3.2 参数敏感性

分析不同 K 值（近邻个数）对分类结果的影响。（待完成：给出不同 K 值下的分类准确率变化情况。）

7.3.3 特征敏感性

分析删除不同化学成分特征对分类结果的影响。（待完成：给出特征敏感性分析的具体结果。）

八、 问题四：化学成分关联关系分析

8.1 分析方法

8.1.1 相关性分析

分别针对高钾玻璃和铅钡玻璃两类，计算各化学成分之间的 Pearson 相关系数：

$$r_{pq} = \frac{\sum_{i=1}^n (x_{ip} - \bar{x}_p)(x_{iq} - \bar{x}_q)}{\sqrt{\sum_{i=1}^n (x_{ip} - \bar{x}_p)^2 \sum_{i=1}^n (x_{iq} - \bar{x}_q)^2}} \quad (8)$$

绘制相关性热力图，可视化展示成分间的关联关系。

8.1.2 对应分析

采用对应分析（Correspondence Analysis）方法，将高维化学成分数据降维到低维空间，可视化展示成分与样本之间的关系。对应分析能够揭示化学成分之间的内在关联结构以及不同类别玻璃在成分空间中的分布特征。

8.2 关联关系分析结果

（待完成：给出高钾玻璃各化学成分间的相关系数矩阵及热力图。）

（待完成：给出铅钡玻璃各化学成分间的相关系数矩阵及热力图。）

（待完成：给出对应分析的降维结果及可视化图。）

8.3 类别间差异性比较

8.3.1 关联强度比较

比较高钾玻璃与铅钡玻璃在化学成分关联关系上的差异。（待完成：计算两类玻璃的平均相关系数，比较关联强度的差异。）

8.3.2 关联模式比较

比较两类玻璃在化学成分关联模式上的差异。（待完成：识别高钾玻璃中关联较强的成分对与铅钡玻璃中关联较强的成分对，分析差异的原因。）

九、 模型评价与改进

9.1 模型优点

1. 采用 CLR 变换处理成分数据，有效消除了定和约束带来的伪相关影响。
2. 综合运用多种统计与机器学习方法，从不同角度验证分析结果的可靠性。

3. K-means++ 算法优化了初始聚类中心的选择，提高了聚类结果的稳定性。
4. 模型体系完整，覆盖了从数据预处理到结果分析的全流程。

9.2 模型不足

1. 样本量相对有限，可能影响统计检验的效能。
2. 风化过程的机理模型尚未充分考虑，预测主要基于统计规律。
3. 部分化学成分数据存在较多缺失值，可能引入偏差。

9.3 改进方向

1. 可引入 GA-BP 神经网络等深度学习模型提高预测精度。
2. 可结合风化机理建立更精细的物理化学模型。
3. 可采用更先进的数据填补方法处理缺失值问题。

十、参考文献

参考文献

- [1] 2022 年高教社杯全国大学生数学建模竞赛 C 题：古代玻璃制品的成分分析与鉴别.
- [2] 古代玻璃制品成分分析与鉴别的统计建模，数学建模及其应用，2023 年 02 期.
- [3] 基于 CLR 的玻璃文物成分分析与分类模型，数学建模及其应用，2023 年 02 期.
- [4] 基于 GA-BP 的古代玻璃类别预测与风化程度测定模型研究，软件工程，2023 年 07 期.
- [5] 玻璃文物成分分析与鉴别的数学模型，黑龙江科学，2024 年 04 期.
- [6] 基于多元线性回归的古代玻璃体系判别方法，甘肃科技，2024 年 02 期.

附录 A 核心代码 (Python)

以下代码根据公开资料整理，用于实现本文各问题的建模与计算。

1.1 数据预处理代码

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4
5 # 读取数据
6 df_info = pd.read_excel('附件.xlsx', sheet_name='表单1')
7 df_comp = pd.read_excel('附件.xlsx', sheet_name='表单2')
8 df_unknown = pd.read_excel('附件.xlsx', sheet_name='表单3')
9
10 # 数据清洗：删除无效数据（累加和不在85%-105%之间）
11 comp_cols = ['二氧化硅(SiO2)', '氧化钠(Na2O)', '氧化钾(K2O)', '氧化钙(CaO)',
12             '氧化镁(MgO)', '氧化铝(Al2O3)', '氧化铁(Fe2O3)', '氧化铜(CuO)',
13             '氧化铅(PbO)', '氧化钡(BaO)', '五氧化二磷(P2O5)', '氧化锶(SrO)',
14             '氧化锡(SnO2)', '二氧化硫(SO2)']
15 df_comp['sum'] = df_comp[comp_cols].fillna(0).sum(axis=1)
16 df_comp_valid = df_comp[(df_comp['sum'] >= 85) & (df_comp['sum'] <= 105)]
17
18 # 空值填0处理
19 df_comp_valid = df_comp_valid.fillna(0)
```

Listing 1: 数据预处理

1.2 CLR 变换代码

```
1 from scipy.stats import gmean
2
3 def clr_transform(data):
4     """中心对数比变换"""
5     # 确保数据为正
6     data_pos = data.copy()
7     data_pos[data_pos <= 0] = 1e-10
8     # 计算几何均值
9     geo_mean = gmean(data_pos, axis=1)
10    # CLR变换
11    clr_data = np.log(data_pos / geo_mean[:, np.newaxis])
12    return clr_data
13
14 # 对有效数据进行CLR变换
15 X_valid = df_comp_valid[comp_cols].values
```

```
16 X_clr = clr_transform(X_valid)
```

Listing 2: 中心对数比变换

1.3 问题一：卡方检验代码

```
1 from scipy.stats import chi2_contingency
2
3 # 合并数据
4 df_merged = pd.merge(df_info, df_comp_valid, left_on='文物编号', right_on='文物采样点',
5                       how='inner')
6
7 # 卡方检验：类型与风化
8 contingency_type = pd.crosstab(df_merged['类型'], df_merged['表面风化'])
9 chi2, p, dof, expected = chi2_contingency(contingency_type)
10 print(f"类型与风化: chi2={chi2:.4f}, p={p:.4f}")
11
12 # 卡方检验：纹饰与风化
13 contingency_deco = pd.crosstab(df_merged['纹饰'], df_merged['表面风化'])
14 chi2, p, dof, expected = chi2_contingency(contingency_deco)
15 print(f"纹饰与风化: chi2={chi2:.4f}, p={p:.4f}")
16
17 # 卡方检验：颜色与风化
18 contingency_color = pd.crosstab(df_merged['颜色'], df_merged['表面风化'])
19 chi2, p, dof, expected = chi2_contingency(contingency_color)
20 print(f"颜色与风化: chi2={chi2:.4f}, p={p:.4f}")
```

Listing 3: 卡方检验

1.4 问题二：K-means++ 聚类代码

```
1 from sklearn.cluster import KMeans
2 from sklearn.metrics import silhouette_score
3
4 def find_optimal_k(X, max_k=10):
5     """肘部法则确定最优K值"""
6     sse = []
7     sil_scores = []
8     for k in range(2, max_k + 1):
9         kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
10        kmeans.fit(X)
11        sse.append(kmeans.inertia_)
12        sil_scores.append(silhouette_score(X, kmeans.labels_))
13    return sse, sil_scores
```

```

14
15 # 高钾玻璃聚类
16 df_gaojia = df_merged[df_merged['类型'] == '高钾']
17 features_gj = ['氧化钾(K2O)', '二氧化硅(SiO2)', '氧化铝(Al2O3)', '氧化钙(CaO)']
18 X_gj = df_gaojia[features_gj].values
19 X_gj_scaled = StandardScaler().fit_transform(X_gj)
20
21 sse_gj, sil_gj = find_optimal_k(X_gj_scaled)
22 # 选择最优K值（根据肘部法则和轮廓系数）
23 kmeans_gj = KMeans(n_clusters=3, init='k-means++', random_state=42, n_init=10)
24 labels_gj = kmeans_gj.fit_predict(X_gj_scaled)
25
26 # 铅钡玻璃聚类
27 df_qianbei = df_merged[df_merged['类型'] == '铅钡']
28 features_qb = ['氧化铅(PbO)', '氧化钡(BaO)', '二氧化硅(SiO2)', '五氧化二磷(P2O5)']
29 X_qb = df_qianbei[features_qb].values
30 X_qb_scaled = StandardScaler().fit_transform(X_qb)
31
32 sse_qb, sil_qb = find_optimal_k(X_qb_scaled)
33 kmeans_qb = KMeans(n_clusters=3, init='k-means++', random_state=42, n_init=10)
34 labels_qb = kmeans_qb.fit_predict(X_qb_scaled)

```

Listing 4: K-means++ 聚类

1.5 问题三：KNN 分类代码

```

1 from sklearn.model_selection import train_test_split, cross_val_score
2 from sklearn.neighbors import KNeighborsClassifier
3 from sklearn.metrics import classification_report, accuracy_score
4
5 # 准备训练数据
6 df_train = df_merged.dropna(subset=['类型'])
7 X_train_all = df_train[comp_cols].fillna(0).values
8 y_train_all = df_train['类型'].values
9
10 # 划分训练集和测试集
11 X_train, X_test, y_train, y_test = train_test_split(
12     X_train_all, y_train_all, test_size=0.2, random_state=42
13 )
14
15 # KNN分类器 (K=1)
16 knn = KNeighborsClassifier(n_neighbors=1)
17 knn.fit(X_train, y_train)
18
19 # 测试集评估
20 y_pred = knn.predict(X_test)

```

```

21 print(classification_report(y_test, y_pred))
22 print(f"准确率: {accuracy_score(y_test, y_pred):.4f}")
23
24 # 交叉验证
25 cv_scores = cross_val_score(knn, X_train_all, y_train_all, cv=5)
26 print(f"5折交叉验证平均准确率: {cv_scores.mean():.4f}")
27
28 # 预测未知样本
29 X_unknown = df_unknown[comp_cols].fillna(0).values
30 y_unknown_pred = knn.predict(X_unknown)
31 print("未知样本预测结果:", y_unknown_pred)
32
33 # 参数敏感性分析 (不同K值)
34 for k in [1, 3, 5, 7, 9]:
35     knn_k = KNeighborsClassifier(n_neighbors=k)
36     scores = cross_val_score(knn_k, X_train_all, y_train_all, cv=5)
37     print(f"K={k}, 平均准确率: {scores.mean():.4f}")

```

Listing 5: KNN 分类

1.6 问题四：相关分析与对应分析代码

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 from sklearn.decomposition import PCA
4
5 # 相关性分析
6 def correlation_analysis(df, glass_type):
7     """分析指定类型玻璃的化学成分相关性"""
8     df_type = df[df['类型'] == glass_type]
9     X_type = df_type[comp_cols].fillna(0)
10    corr_matrix = X_type.corr()
11
12    # 绘制热力图
13    plt.figure(figsize=(12, 10))
14    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, fmt='.2f')
15    plt.title(f'{glass_type}玻璃化学成分相关系数矩阵')
16    plt.show()
17    return corr_matrix
18
19 # 高钾玻璃相关性
20 corr_gj = correlation_analysis(df_merged, '高钾')
21
22 # 铅钡玻璃相关性
23 corr_qb = correlation_analysis(df_merged, '铅钡')
24
25 # 对应分析 (使用PCA降维可视化)
26 def correspondence_analysis(df):

```



```

25     """对应分析：将成分数据降维到2D空间"""
26     X = df[comp_cols].fillna(0).values
27     X_clr = clr_transform(X)
28     pca = PCA(n_components=2)
29     X_pca = pca.fit_transform(X_clr)
30     plt.figure(figsize=(10, 8))
31     colors = {'高钾': 'red', '铅钒': 'blue'}
32     for glass_type in ['高钾', '铅钒']:
33         mask = df['类型'] == glass_type
34         plt.scatter(X_pca[mask, 0], X_pca[mask, 1],
35                    c=colors[glass_type], label=glass_type, alpha=0.7)
36     plt.xlabel('第一主成分')
37     plt.ylabel('第二主成分')
38     plt.title('玻璃样品对应分析二维投影')
39     plt.legend()
40     plt.show()
41     return X_pca
42
43 X_pca = correspondence_analysis(df_merged.dropna(subset=['类型']))

```

Listing 6: 相关分析与对应分析

1.7 风化前成分预测代码

```

1  from sklearn.linear_model import LinearRegression
2  from sklearn.neighbors import KNeighborsRegressor
3
4  def predict_weathered_components(df, weathered_samples, method='knn'):
5      """
6      预测风化样本的风化前成分含量
7      method: 'weighted_avg', 'linear', 'knn'
8      """
9      results = {}
10     for idx, sample in weathered_samples.iterrows():
11         # 找到同类型未风化样本
12         same_type = df[df['类型'] == sample['类型']]
13         unweathered = same_type[same_type['表面风化'] == '无风化']
14
15         if method == 'weighted_avg':
16             # 加权平均法
17             weights = 1 / np.abs(unweathered[comp_cols].fillna(0) -
18                                sample[comp_cols].fillna(0)).sum(axis=1)
19             weights = weights / weights.sum()
20             pred = np.average(unweathered[comp_cols].fillna(0), weights=weights, axis=0)
21
22         elif method == 'linear':

```

```

23     # 多元线性回归
24     X_train = unweathered[comp_cols].fillna(0).values
25     pred = []
26     for col in comp_cols:
27         model = LinearRegression()
28         X_other = np.delete(X_train, comp_cols.index(col), axis=1)
29         y_train = unweathered[col].fillna(0).values
30         model.fit(X_other, y_train)
31         x_test = np.delete(sample[comp_cols].fillna(0).values,
32                             comp_cols.index(col))
33         pred.append(model.predict([x_test])[0])
34
35     elif method == 'knn':
36         # K近邻回归
37         X_train = unweathered[comp_cols].fillna(0).values
38         y_train = unweathered[comp_cols].fillna(0).values
39         knn = KNeighborsRegressor(n_neighbors=5)
40         knn.fit(X_train, y_train)
41         pred = knn.predict([sample[comp_cols].fillna(0).values])[0]
42
43     results[sample['文物采样点']] = pred
44     return results
45
46 # 预测风化点
47 weathered_samples = df_merged[df_merged['表面风化'] == '风化']
48 predictions = predict_weathered_components(df_merged, weathered_samples, method='knn')

```

Listing 7: 风化前成分预测